eventos

Los agregados registran eventos de dominio y Spring los publica al guardarlos (@DomainEvents). Los oyentes corren en la misma transacción: la bandeja quita la donación que salió del depósito, las necesidades reciben lo entregado y los avisos se escriben en la bandeja de salida (no saben si la transacción no confirma).

BandejaDeCandidatosOyente
- private candidatos: RepositorioCandidatosAsignacion
+ public alCambiarDeEstado(evento): DonacionCambioDeEstado(s): void
+ public alDarseDeBaja(evento: DonacionDadaDeBaja(s): void

NecesidadesOyente
- private entidades: RepositorioEntidadesBeneficiarias
+ public alEntregarse(evento: DonacionEntregada): void

AvisosOyente
- private donaciones: RepositorioDonaciones
- private donantes: RepositorioPersonasDonantes
- private entidades: RepositorioEntidadesBeneficiarias
- private notificador: Notificador
- private incentivos: IncentivosClient
+ public alAsignarse(evento: DonacionAsignada): void «antes del commit»
+ public alEntregarse(evento: DonacionEntregada): void «antes del commit»
+ public alRegistrarse(evento: DonacionesRegistradas): void «antes del commit»

services

logistica

Un manejador por aviso del broker. Los avisos son idempotentes: uno repetido responde 200 y no cambia el estado ni avisa de nuevo. Donantes y entidades de un lote se leen en dos consultas.

«@Transactional» ManejadorInicioDeRuta
- private donaciones: RepositorioDonaciones
- private participantes: ParticipantesDeDonaciones
- private notificador: Notificador
+ public procesar(dto: InicioRutaCallbackDTO): void

ParticipantesDeDonaciones
- private donantes: RepositorioPersonasDonantes
- private entidades: RepositorioEntidadesBeneficiarias
+ public donanteDe(donacion: Donacion): PersonaDonante
+ public entidadDe(donacion: Donacion): EntidadBeneficiaria
+ public de(donaciones: List<Donacion>): Lote

«record» ParticipantesDeDonaciones.Lote
donantes: Map<UUID, PersonaDonante>
entidades: Map<UUID, EntidadBeneficiaria>

«@Transactional» ManejadorEntregaExitosa
- private donaciones: RepositorioDonaciones
- private validador: DonacionesValidador
- private participantes: ParticipantesDeDonaciones
- private notificador: Notificador
+ public procesar(dto: EntregaExitosaCallbackDTO): void

«@Transactional» ManejadorEntregaFallida
- private donaciones: RepositorioDonaciones
- private validador: DonacionesValidador
- private participantes: ParticipantesDeDonaciones
- private notificador: Notificador
+ public procesar(dto: EntregaFallidaCallbackDTO): void

AsignacionBatchService
- private donacionesRepositorio: RepositorioDonaciones
- private asignacionService: AsignacionDonacionesService
- private candidatosRepositorio: RepositorioCandidatosAsignacion
- private entidadesRepositorio: RepositorioEntidadesBeneficiarias
- private notificador: NotificacionTransport
- private transaccion: TransactionOperations
+ public ejecutarMatchmaking(fechorario): void «@Scheduled 0 30 3 * * *, @SchedulerLock»

AsignacionDonacionesService
- private algoritmos: List<AlgoritmoAsignacion>
- private entidadesRepository: RepositorioEntidadesBeneficiarias
- private tamanioRanking: int
+ public generarRanking(donacion: Donacion): ResultadoMatchmaking
+ public generarRanking(donacion: Donacion, entidades: List<EntidadBeneficiaria>): ResultadoMatchmaking

Algoritmos enchufables: recibe todos los beans AlgoritmoAsignacion (en su @Order) y devuelve el ranking de cada uno por su nombre. Sumar un algoritmo no toca esta clase, ni el esquema, ni el JSON.

AlgoritmoPrioridadSubAtendidos
- public NOMBRE: String = "PRIORIDAD_SUBATENDIDOS"
- private MUJOL_TAMAÑO_TIRO: int = 2
- private readonly repositorioRepositorioEntidad

+ public nombre(): String
+ protected prepararPuntaje(donacion: Donacion, entidades: List<EntidadBeneficiaria>): ToDoubleFunction<EntidadBeneficiaria>
+ protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double

«abstract» AlgoritmoAsignacionBase
+ public final evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>, tamanio: int): List<ResultadoAsignacion>
protected prepararPuntaje(donacion: Donacion, entidades: List<EntidadBeneficiaria>): ToDoubleFunction<EntidadBeneficiaria>
protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
protected incluir(puntaje: double): boolean

AlgoritmoCompatibilidadSemantica
- public NOMBRE: String = "COMPATIBILIDAD_SEMANTICA"
+ public nombre(): String
protected calcularPuntaje(donacion: Donacion, entidad: EntidadBeneficiaria): double
protected incluir(puntaje: double): boolean

«@Transactional» DonacionComandosService
- private repositorio: RepositorioDonaciones
- private mapper: DonacionMapper
- private validador: DonacionesValidador
- private entidadesValidador: EntidadesBeneficiariasValidator
+ public cambiarEstado(id: UUID, dto: CambioEstadoRequestDTO): void
+ public modificarBien(id: UUID, idBien: UUID, dto: BienRequestDTO): DonacionResponseDTO
+ public modificarUnicoBien(id: UUID, dto: BienRequestDTO): void
+ public asignar(id:Donacion: UUID, dto: AsignacionRequestDTO): void
+ public eliminar(id: UUID): void

Comandos y consultas separadas: los comandos cambian un agregado y lo guardan (sus eventos se publican al guardarlo); las consultas leen y arman DTOs dentro de una transacción de solo lectura (las colecciones se cargan al usarse).

«@Transactional(readOnly)» DonacionConsultasService
- private repositorio: RepositorioDonaciones
- private mapper: DonacionMapper
- private asignacionService: AsignacionDonacionesService
- private entidadesRepositorio: RepositorioEntidadesBeneficiarias
- private entidadMapper: EntidadBeneficiariaMapper
- private validador: DonacionesValidador
- private candidatosRepositorio: RepositorioCandidatosAsignacion

+ public obtenerDonaciones(filtro: FiltroDonaciones, pagina: SolicitudPagina): PaginaResponseDTO<DonacionResponseDTO>
+ public obtenerPorId(id: UUID): DonacionResponseDTO
+ public obtenerCandidatos(idDonacion: UUID): CandidatosAsignacionResponseDTO
+ public obtenerCandidatosPendientes(): List<CandidatosPendientesResponseDTO>

PersonaDonanteService.eliminar y EntidadesBeneficiariasService.eliminarEntidad rechazan con 409 si borrado si hay donaciones (PersonaConDonacionesException / EntidadConDonacionesException). Una persona BLOQUEADA no registra donaciones (422).

Cada lote de 500 filas es una transacción. Busca los emails del lote en una consulta (obtenerPorEmails) y guarda con guardarTodos (inserts en lotes de 50). Si el lote falla, lo reprocesa fila por fila para informar el error exacto. La respuesta resume y detalla hasta 100 errores.

«@Transactional» SegmentadoDonacionesService
- private donacionesRepository: RepositorioDonaciones
- private donantesRepository: RepositorioPersonasDonantes
- private eventos: ApplicationEventPublisher
- private personasValidador: PersonasValidador
- private mapper: DonacionMapper

+ public registrar(dto: DonacionRequestDTO): List<DonacionResponseDTO>

«@Transactional» EntidadesBeneficiariasService
- private repositorio: RepositorioEntidadesBeneficiarias
- private donaciones: RepositorioDonaciones
- private validador: EntidadesBeneficiariasValidator
- private mapper: EntidadBeneficiariaMapper
- private direcciones: DireccionMapper
- private contactos: ContactoMapper
- private representantes: RepresentanteMapper

+ public guardar(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiariaResponseDTO
+ public obtenerEntidades(pagina: SolicitudPagina): PaginaResponseDTO<EntidadBeneficiariaResponseDTO>
+ public obtenerPorId(id: UUID): EntidadBeneficiariaResponseDTO
+ public actualizar(id: UUID, dto: EntidadBeneficiariaRequestDTO): void
+ public eliminar(id:Entidad: UUID): void
+ public agregarCampania(idEntidad: UUID, dto: CampaniaRequestDTO): CampaniaResponseDTO
+ public agregarNecesidad(idCampania:Entidad: UUID, campaniaId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
+ public obtenerNecesidadesDeCampania(entidadId: UUID, campaniaId: UUID): List<NecesidadResponseDTO>
+ public actualizarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID): NecesidadResponseDTO
+ public actualizarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID, dto: NecesidadRequestDTO): NecesidadResponseDTO
+ public eliminarNecesidad(entidadId: UUID, campaniaId: UUID, necesidadId: UUID): void

«@Transactional» PersonaDonanteService
- private repositorio: RepositorioPersonasDonantes
- private donaciones: RepositorioDonaciones
- private validador: PersonasValidador
- private mapper: PersonaDonanteMapper
- private direcciones: DireccionMapper
- private contactos: ContactoMapper
- private representantes: RepresentanteMapper

+ public registrar(dto: PersonaDonanteRequestDTO): PersonaDonanteResponseDTO
+ public obtenerDonante(id: UUID): PersonaDonanteResponseDTO
+ public obtenerDonantes(estado: String, pagina: SolicitudPagina): PaginaResponseDTO<PersonaDonanteResponseDTO>
+ public actualizar(id: UUID, dto: PersonaDonanteRequestDTO): void
+ public modificarContacto(id: UUID, dto: MedioDeContactoRequestDTO): void
+ public modificarRepresentante(id: UUID, dto: RepresentanteRequestDTO): void
+ public actualizar(id: UUID, dto: PersonaDonanteUpdateRequestDTO): PersonaDonanteResponseDTO
+ public eliminar(id: UUID): void

InactividadDonantesService
- private donanteRepository: RepositorioPersonasDonantes
- private notificador: Notificador
- private propiedades: DonacionesProperties

+ public notificarDonantesInactivos(): void «@Scheduled 0 0 0 * * *, @SchedulerLock, @Transactional»

ReconciliacionRecepcionesService
- private recepciones: RepositorioRecepcionesEntidad

+ public reconciliar(): void «@Scheduled 0 0 3 * * *, @SchedulerLock, @Transactional»

Template method: evaluar() puntúa, filtra, ordena y recorta al tamaño configurado (donaciones.matchmaking.tamanio-ranking). SubAtendidos redefine prepararPuntaje para leer en UNA consulta las recepciones del trimestre de todas las entidades (salta entidad_recepcion_diaria).

Los procesos nocturnos recorren las donaciones de a lotes de 100 ordenados por id (obtenerLoteEnEstado), cada uno en una transacción corta. El despacho manda cada lote al broker por separado y deja la llamada fuera de la transacción.
Todos los cron usan la zona America/Argentina/Buenos_Aires (FechaHoraArgentina.ZONA_ID), no la del servidor.

PlanificacionEntregasService
- private donacionesRepository: RepositorioDonaciones
- private entidadesRepository: RepositorioEntidadesBeneficiarias
- private brokerClient: BrokerLogisticaClient
- private notificador: NotificacionTransport
- private transaccion: TransactionOperations

+ public despacharSolicitudes(donaciones: List<UUID>, proveedor: ProveedorLogistica): EnvioLogisticaResponseDTO
+ public despacharPendientesDeDia(): void «@Scheduled 0 0 4 * * *, @SchedulerLock»

CsvImportService
- private donanteRepository: RepositorioPersonasDonantes
- private parser: DonanteCsvRowParser
- private factory: DonanteFactory
- private propiedades: DonacionesProperties
- private transaccion: TransactionOperations

+ public importar(YResumicipcesStream: InputStream): ImportacionResumenResponseDTO
+ public importar(csvStream: InputStream): ImportReport

validadores

EntidadesBeneficiariasValidator
- private repositorio: RepositorioEntidadesBeneficiarias

+ public validarYObtenerEntidad(id: UUID): EntidadBeneficiaria
+ public validarYObtenerCampania(entidad: EntidadBeneficiaria, campaniaId: UUID): Campania
+ public validarYObtenerNecesidad(entidad: EntidadBeneficiaria, campania: Campania, necesidadId: UUID): Necesidad
- public validarFechasCampania(inicio: LocalDate, fin: LocalDate): void

DonacionesValidador
- private repository: RepositorioDonaciones

+ public validarYObtenerDonacion(id: UUID): Donacion

PersonasValidador
- private repositorio: RepositorioPersonasDonantes

+ public validarExistencia(id: UUID): PersonaDonante
+ public validarYObtenerEntidad(id: UUID): PersonaJuridica
+ public validarMedioDeContacto(medio: MedioDeContacto): void

clientes

IncentivosClient
- private salida: BandejaDeSalida

+ public notificarDonacionRegistrada(donanteId, destinatario, medio, cantidadBienes, categoriasDonadas, fecha): void
+ public notificarDonacionEditada(donacion: Donacion, destinatario: String, medio: String, fechaEntrega: LocalDateTime): void

«record» SolicitudEnvio
- proveedor: String
- donaciones: List<DonacionEnvio>

BrokerLogisticaClient.Api
+ public solicitarEnvio(pedido: SolicitudEnvio): ResultadoEnvio «@PostExchange /api/broker/envios»

BrokerLogisticaClient
- private api: Api

+ public solicitarEnvio(proveedor: String, donaciones: List<DonacionEnvio>): ResultadoEnvio

«record» DonacionEnvio
- idDonacion: UUID
- idEntidadBeneficiaria: UUID
- nombreEntidad: String
- direccionEntrega: DireccionEnvio

«record» ResultadoEnvio
- proveedor: String
- envioAsignados: List<EnvioAsignado>
- errores: List<ErrorAsignacion>

«record» EnvioAsignado
- idDonacion: UUID
- proveedor: String
- idRequerimiento: String
- fechaAsignacion: LocalDateTime

«record» DireccionEnvio
- calle: String
- numero: int
- localidad: String
- provincia: String
- codigoPostal: String

NotificacionClient e IncentivosClient no llaman a nadie: encolan el aviso en la bandeja de salida. BrokerLogisticaClient llama directo al broker (servicio-broker-logistica, 8087): el despacho necesita la respuesta.

NotificacionClient
- private salida: BandejaDeSalida

+ public enviarNotificacion(destinatario: String, mensaje: String, medio: String): void

mappers

DonacionMapper
+ public toDTO(donacion: Donacion): DonacionResponseDTO
+ public toDTO(Donaciones: List<Donacion>): List<DonacionResponseDTO>
+ public toDonacion(dto: DonacionRequestDTO): CargaDonacion
+ public toBien(dto: BienRequestDTO): Bien

EntidadBeneficiariaMapper
- private direcciones: DireccionMapper
- private contactos: ContactoMapper
- private representantes: RepresentanteMapper

+ public toModel(dto: EntidadBeneficiariaRequestDTO): EntidadBeneficiaria
+ public toDTO(entidad: EntidadBeneficiaria): EntidadBeneficiariaResponseDTO
+ public toCampaniaModel(dto: CampaniaRequestDTO, idEntidad: UUID): Campania
+ public toCampaniaDTO(campania: Campania): CampaniaResponseDTO
+ public toNecesidadDTO(necesidad: Necesidad): NecesidadResponseDTO

DireccionMapper
+ public toDireccion(dto: DireccionRequestDTO): Direccion
+ public toDireccionDTO(direccion: Direccion): DireccionResponseDTO

PersonaDonanteMapper
- private direcciones: DireccionMapper
- private contactos: ContactoMapper
- private representantes: RepresentanteMapper

+ public toModel(dto: PersonaDonanteRequestDTO): PersonaDonante
+ public toDTO(persona: PersonaDonante): PersonaDonanteResponseDTO

RepresentanteMapper
- private contactos: ContactoMapper

+ public toRepresentante(dto: RepresentanteRequestDTO): Representante
+ public toRepresentantes(dtos: List<RepresentanteRequestDTO>): List<Representante>
+ public toRepresentanteDTO(representante: Representante): RepresentanteResponseDTO
+ public toRepresentantesDTO(representantes: List<Representante>): List<RepresentanteResponseDTO>

ContactoMapper
+ public toContacto(dto: MedioDeContactoRequestDTO): MedioDeContacto
+ public toContactos(dtos: List<MedioDeContactoRequestDTO>): List<MedioDeContacto>
+ public toContactoDTO(contacto: MedioDeContacto): MedioDeContactoResponseDTO
+ public toContactosDTO(contactos: List<MedioDeContacto>): List<MedioDeContactoResponseDTO>

importacion

DonanteFactory
+ public crearPersona(dto: DonanteImportDto): PersonaDonante

ImportReport
- private resultados: List<ImportFilaCSV>

+ public agregar(result: ImportFilaCSV): void
+ public totalCreados(): long
+ public totalActualizados(): long
+ public getErrores(): List<ImportFilaCSV>
+ public totalProcesados(): int

ImportFilaCSV
- private numeroLinea: int
- private estado: EstadoImport
- private email: String
- private detalle: String

+ public creado(linea: int, email: String): ImportFilaCSV
+ public actualizado(linea: int, email: String): ImportFilaCSV
+ public error(linea: int, email: String, motivo: String): ImportFilaCSV

«enum» EstadoImport
CREADO
ACTUALIZADO
ERROR

Valida cada fila antes de procesarla: el email con su forma, el documento con su tipo (Documento.de) y que una jurídica tenga CUIT. Una fila inválida se informa con su línea.

DonanteCsvRowParser
+ public parsear(columnas: String[], numeroLinea: int): DonanteImportDto

«record» DonanteImportDto
- linea: int
- tipoPersona: String
- nombre: String
- documento: String
- nombreORazonSocial: String
- email: String
- telefono: String

notificaciones

Arma el texto de cada aviso y elige el canal: el contacto predeterminado de la persona o, si no tiene, su email; para la entidad, su email o su primer contacto. Las entregas fallidas también avisan a la persona administradora.

Notificador
- private cliente: NotificacionClient
- private propiedades: DonacionesProperties

+ public destinoDe(donante: PersonaDonante): Optional<Destino>
+ public destinoDe(entidad: EntidadBeneficiaria): Optional<Destino>
+ public donacionAsignada(donante: PersonaDonante, entidad: EntidadBeneficiaria): void
+ public entregaProgramada(donante: PersonaDonante, entidad: EntidadBeneficiaria, cuando: String): void
+ public entregaEnCurso(donante: PersonaDonante, entidad: EntidadBeneficiaria, cuando: String): void
+ public entregaFallida(idDonacion: UUID, donante, entidad, motivo: String, reprogramada: boolean): void
+ public inactividad(donante: PersonaDonante): boolean

«record» Destino
- valor: String
- canal: Canal

interfaces

services

AlgoritmoAsignacion
+ public nombre(): String
+ public evaluar(donacion: Donacion, entidades: List<EntidadBeneficiaria>, tamanio: int): List<ResultadosAsignacion>

1 ▷

integracion

NotificacionTransport
+ public enviar(solicitud: SolicitudNotificacionMensaje): void
+ public nombre(): String

repositorios

RepositorioDonaciones
+ public cargarDonaciones(donaciones: List<Donacion>): void
+ public guardar(donacion: Donacion): void
+ public obtenerLoteEnEstado(estado: String, desquesDe: UUID, cantidad: int): List<Donacion>
+ public buscar(filtro: FiltroDonaciones, pagina: SolicitudPagina): Pagina<Donacion>
+ public buscarPorId(id: UUID): Optional<Donacion>
+ public obtenerPorIds(ids: Collection<UUID>): List<Donacion>
+ public existenDeDonante(idDonante: UUID): boolean
+ public existenAsignadasA(idEntidad: UUID): boolean

RepositorioEntidadesBeneficiarias
+ public guardar(entidad: EntidadBeneficiaria): void
+ public buscarTodas(): List<EntidadBeneficiaria>
+ public buscar(pagina: SolicitudPagina): Pagina<EntidadBeneficiaria>
+ public buscarPorId(id: UUID): Optional<EntidadBeneficiaria>
+ public obtenerPorIds(ids: Collection<UUID>): List<EntidadBeneficiaria>
+ public eliminar(entidad: EntidadBeneficiaria): void

RepositorioRecepcionesEntidad
+ public contarRecepcionesPorEntidadDesde(desde: LocalDate): Map<UUID, Long>
+ public contarRecepcionesDesde(idEntidad: UUID, desde: LocalDate): long
+ public reconciliar(): int

RepositorioPersonasDonantes
+ public buscar(petado: String, pagina: SolicitudPagina): Pagina<PersonaDonante>
+ public obtenerParaAvisoDeEntrega(idDonante: UUID): List<PersonaDonante>
+ public registrarAvisoDeInactividad(ids: Collection<UUID>, cuando: LocalDateTime): void
+ public guardar(donante: PersonaDonante): void
+ public guardarTodos(donantes: Collection<PersonaDonante>): void
+ public existePorEmail(email: String): boolean
+ public buscarPorId(id: UUID): Optional<PersonaDonante>
+ public obtenerPorIds(ids: Collection<UUID>): List<PersonaDonante>
+ public buscarPorEmail(email: String): Optional<PersonaDonante>
+ public obtenerPorEmails(emails: Collection<String>): Map<String, PersonaDonante>
+ public eliminar(persona: PersonaDonante): void

RepositorioCandidatosAsignacion
+ public reemplazarTodos(rankings: Collection<RankingPrecalculado>): void
+ public obtenerTodos(): List<RankingPrecalculado>
+ public eliminar(idDonacion: UUID): void

desde()

util

NormalizadorTexto
+ public normalizar(texto: String): String

NormalizadorEmail
+ public normalizar(email: String): String
+ public esValido(email: String): boolean

«record» SolicitudPagina
- public TAMANO_POR_DEFECTO: int = 20
- public TAMANO_MAXIMO: int = 100
- numero: int
- tamanio: int

+ public de(numero: Integer, tamanio: Integer): SolicitudPagina

SolicitudPagina valida en su constructor: pagina >= 0 y 1 <= size <= 100 (dio): sin valor, 0 y 20).
si no, PaginaInvalidaException → 400.
NormalizadorEmail: trim + NFC + minúsculas (Locale.ROOT).
NormalizadorTexto: minúsculas, sin acentos y con los espacios colapsados (subcategorías y matching).
FechaHoraArgentina: el reloj se puede reemplazar en los tests. Donaciones, donantes y entidades responden sus listados con PaginaResponseDTO.desde(Pagina).

FechaHoraArgentina
- public ZONA_ID: String = "America/Argentina/Buenos_Aires"
- private ZONA: ZoneId

+ public ahora(): LocalDateTime
+ public hoy(): LocalDate
+ public usarReloj(reloj: Clock): void
+ public restaurarZonaReloj(): void

«record» FiltroDonaciones
- estado: String
- idDonante: UUID
- categoria: String
- idEntidad: UUID
- soloConBienesVencidos: boolean

«record» Pagina
- contenido: List<T>
- numero: int
- tamanio: int
- totalElementos: long

+ public totalPaginas(): int
+ public map(transformacion: Function<T, R>): Pagina<R>

dtos

responses

PaginaResponseDTO
- private contenido: List<T>
- private pagina: int
- private tamanio: int
- private totalElementos: long
- private totalPaginas: int

+ public desde(pagina: Pagina<T>): PaginaResponseDTO<T>

integracion

notificaciones

«record» SolicitudNotificacionMensaje
- destinatario: String
- mensaje: String
- medio: String

NotificacionTransportAmqp
- private rabbitTemplate: RabbitTemplate

+ public enviar(solicitud: SolicitudNotificacionMensaje): void
+ public nombre(): String

NotificacionTransportRest
- private api: Api

+ public enviar(solicitud: SolicitudNotificacionMensaje): void
+ public nombre(): String

NotificacionTransportRest.Api
+ public enviar(solicitud: SolicitudNotificacionMensaje): void «@PostExchange /api/notificaciones»

EnviadorNotificaciones
- private transporte: NotificacionTransport
- private json: ObjectMapper

+ public destinos(): Set<String>
+ public enviar(destino: String, payload: String): void

Se crea una sola implementación según la property notificaciones.transporte: rest (predeterminado) → POST /notificaciones (8084) amqp → exchange donatrack.notificaciones, routing key notificacion.enviar; espera la confirmación del broker (hasta 5 s) y si no llega, el relay reintenta.

incentivos

EnviadorIncentivos
- private envios: Map<String, Consumer<String>>

+ public destinos(): Set<String>
+ public enviar(destino: String, payload: String): void

EnviadorIncentivos.Api
+ public donacionRegistrada(solicitud: String): void «@PostExchange /api/incentivos/donantes/donacion»
+ public donacionEditada(solicitud: String): void «@PostExchange /api/incentivos/donantes/donacion-editada»

repositories

Spring Data JPA

EntidadesBeneficiariasJpaRepository
- private ORDEN_LISTADO: Sort = sourceSort ASC, id ASC

+ public entidadesBeneficiariasJpaRepository

«interface» EntidadesBeneficiariasJpaRepository

RecepcionesEntidadRepository
- private jpa: RecepcionDiariaJpaRepository

+ public reconciliar(): int

RecepcionDiariaJpaRepository
+ public totalesDesde(desde: LocalDate): List<TotalPorEntidad> «@Query»
+ public totalDesde(idEntidad: UUID, desde: LocalDate): long «@Query»
+ public sumar(idEntidad: UUID, fecha: LocalDate, delta: long): int «@Modifying»
+ public vaciar(): void «@Modifying nativo»
+ public recalcularDesde(Donaciones): int «@Modifying nativo»

«projection» RecepcionDiariaJpaRepository.TotalPorEntidad
- public getIdEntidad(): UUID
- public getTotal(): Long

DonacionesRepository
- private ORDEN_LISTADO: Sort = fechaDonacion DESC, id ASC
- private jpa: DonacionJpaRepository

+ public guardar(donacion: Donacion): void
+ public obtenerLoteEnEstado(estado: String, desquesDe: UUID, cantidad: int): List<Donacion>
+ public buscar(filtro: FiltroDonaciones, pagina: SolicitudPagina): Pagina<Donacion>
+ private ajustarRecepciones(agregacion: Agregacion, delta: long): void
+ private especificacion(filtro: FiltroDonaciones): Specification<Donacion>

DonacionJpaRepository
+ public findByEstadoInAndIdentidadEntidadIn(estados: String, Pageable): List<Donacion>
+ public findByEstadoInAndIdentidadEntidadIdIn(estados: String, id: UUID, lote: Pageable): List<Donacion>
+ public existsByIdDonante(idDonante: UUID): boolean
+ public existsByIdEntidadBeneficiariaIdEntidadBeneficiaria: UUID): boolean

PersonaDonanteJpaRepository
+ public findByEmail(email: String): Optional<PersonaDonante>
+ public existsByEmail(email: String): boolean
+ public findByEmailIn(emails: Collection<String>): List<PersonaDonante>
+ public findByEstadoInAndIdentidadEntidadIdIn(estados: String, pagina: Pageable): Page<PersonaDonante>
+ public buscarParaAvisoDeInactividad(desde: LocalDateTime, pagina: Pageable): List<PersonaDonante> «@Query»
+ public registrarAvisoDeInactividad(ids: Collection<UUID>, cuando: LocalDateTime): int «@Modifying nativo»

CandidatosAsignacionRepository
- private jpa: RankingPrecalculadoJpaRepository

+ public reemplazarTodos(rankings: Collection<RankingPrecalculado>): void

RankingPrecalculadoJpaRepository
+ public vaciarPuestos(): void «@Modifying nativo»
+ public vaciarRankings(): void «@Modifying nativo»

MensajesSalientesJpaRepository
+ public buscarLatasJesperado: EstadoMensaje, ahora: LocalDateTime, pagina: Pageable): List<MensajeSaliente> «@Query»
+ public countByEstadoIgnestado: EstadoMensaje): long
+ public borrarEnviadosAntesDe(estado: EstadoMensaje, limite: LocalDateTime): int «@Modifying»

«interface» EntidadesBeneficiariasJpaRepository
«interface» RecepcionDiariaEntidadJpaRepository
«interface» DonacionJpaRepository
«interface» DonacionJpaRepository.EspecificationExecutor
«interface» PersonasDonantesJpaRepository
«interface» RankingPrecalculadoJpaRepository
«interface» MensajesSalientesJpaRepository

config

Configuracion tipada y validada al arrancar (prefijo donaciones.): días de inactividad, colores, tamaños de lote, email del administrador, tamaño del ranking y parámetros de la bandeja de salida. Cada valor tiene su default.

«record» DonacionesProperties
- inactividad: Inactividad
- process: Procesos
- importacion: Importacion
- notificaciones: Notificaciones
- matchmaking: Matchmaking
- salida: Salida

BloqueoDeTareasConfig
+ public lockProvider(dataSource: DataSource): LockProvider

RabbitMQConfig
- public EXCHANGE: String = "donatrack.notificaciones"
- public ROUTING_KEY: String = "notificacion.enviar"
- public QUEUE: String = "notificaciones.queue"

«@Configuration»

«@Bean» ClienteHttp
+ public crear(builder: RestClient.Builder, urlBase: String, api: Class<T>): T

«@Bean» CorrelationId
- public ENCABEZADO: String = "X-Correlation-Id"
- public CLAVE_MDC: String = "correlationId"

+ public prepagador(): ClientHttpRequestInterceptor

salida

RelayProgramado
- private relay: RelayDeMensajes

+ public enviarPendientes(): void «@Scheduled cada 5 s, @SchedulerLock»
+ public liberarEnviadosAtascados(): void «@Scheduled 0 30 4 * * *, @SchedulerLock»

RelayDeMensajes
- private mensajes: MensajeSalienteJpaRepository
- private envio: EnviadorDeMensajes
- private transaccion: TransactionOperations
- private configuracion: DonacionesProperties.Salida

+ public enviarPendientes(): int
+ public liberarEnviadosAtascados(): int

EnviadorDeMensajes
+ public destinos(): Set<String>
+ public enviar(destino: String, payload: String): void

Bandeja de salida (outbox): el aviso se guarda en mensaje_saliente en la transacción del cambio. El relay toma los pendientes de a 50, los envía por el enviador de su destino y, si falla, reintenta con una espera que se duplica (30 s hasta 1 h), hasta 10 intentos; después queda FALLIDO. Los enviadores se forman a los 7 días.

«entidad» MensajeSaliente
- private id: UUID
- private destino: String
- private payload: String
- private estado: EstadoMensaje
- private intentos: int
- private creado: LocalDateTime
- private proximoIntento: LocalDateTime
- private enviado: LocalDateTime
- private ultimoError: String

+ public nuevo(destino: String, payload: String, ahora: LocalDateTime): MensajeSaliente
+ public marcarEnviado(ahora: LocalDateTime): void
+ public registrarFallo(error: String, ahora: LocalDateTime, maximoIntentos: int): void

«enum» EstadoMensaje
PENDIENTE
ENVIADO
FALLIDO

BandejaDeSalida
- private mensajes: MensajeSalienteJpaRepository
- private json: ObjectMapper

+ public encolar(destino: String, contenido: Object): void

«create»

Destinos
- public NOTIFICACION: String
- public DIRECCION_REGISTRADA: String
- public INCENTIVOS_DONACION_ENTREGA: String